

Modules and Packages

Dr. Sanjit Kumar Saha

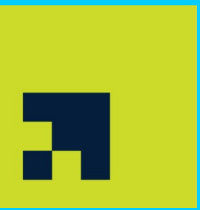


Gisma
University
of Applied
Sciences



Modules

- A module is a file containing Python code that defines functions, classes, and variables.
- It can be imported and used in other Python programs.
- It typically focuses on a specific functionality or set of related functionalities.
- Examples: `math.py`, `random.py`



Module Benefits

[Hunt 2019]

3

- **Simplicity**
 - Focussing on a subset of an overall problem helps us develop solutions that work for the subset and can be combined together to solve the overall problem
- **Maintenance**
 - Modules typically make it easier to define logical boundaries between one body of code and another
- **Testing**
 - As one module can be made independent of another module there are less dependencies and cross overs
- **Reusability**
 - Defining a function or class on one module means that it is easier to reuse that function or class in another module
- **Scoping**
 - Modules typically also define a namespace, that is a scope within which each function or class is unique



Create a Module

- To create a module just save the code you want in a file with the file extension **.py**:

```
def greeting(name):  
    print("Hello, " + name)
```

- Save this code in a file named ***mymodule.py***
- Now we can use the module we just created, by using the ***import*** statement.
- Import the module named ***mymodule***, and call the ***greeting*** function:

```
import mymodule  
  
mymodule.greeting("Jonathan")
```



Variables in Module

- The module can contain functions, as already described, but also variables of all types (arrays, dictionaries, objects etc):

```
person1 = {  
    "name": "John",  
    "age": 36,  
    "country": "Norway"  
}
```

- Save this code in the file ***mymodule.py***
- Import the module named ***mymodule***, and access the ***person1*** dictionary:

```
import mymodule
```

```
a = mymodule.person1["age"]  
print(a)
```



Re-naming a Module

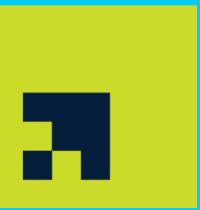
[Hunt 2019]

6

- We can create an alias when we import a module, by using the *as* keyword:

```
import mymodule as mx
```

```
a = mx.person1["age"]  
print(a)
```

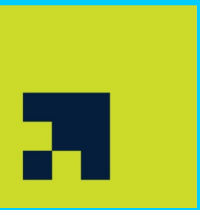


Importing from a Module

[Hunt 2019]

7

- `import utils`
 - While Importing the whole module makes it very clear that these features come from this module, it is a little tedious
 - It is the equivalent of referring to a person via their full name every time we speak to them
 - We could just use their first name
- `from utils import *`
 - A variant of the import statement allows us to import everything from a particular module
 - Which can be read as from the utils module import everything
 - This approach removes the need to prefix the modules functions or classes with the module name
 - `printer(shape)`
 - However, it can result in name clashes as it brings into scope all the elements defined in the imported modules
- `from utils import Shape`
 - We can choose to only bring some features
 - Now, only the Shape class has been imported into the file and made directly available



Import From Module

- We can choose to import only parts from a module, by using the from keyword

```
def greeting(name):  
    print("Hello, " + name)
```

```
person1 = {  
    "name": "John",  
    "age": 36,  
    "country": "Norway"  
}
```

```
from mymodule import person1
```

```
print (person1["age"])
```



Module Properties

- Every module has a set of properties
 - `__name__` the name of the module
 - `__doc__` the docstring for the module
 - `__file__` the file in which the module was defined
 - These properties are considered special as they all start and end with a double underbar (`__`)
- You can also obtain a list of the contents of a module once it has been imported using the `dir()` function
 - `print(dir(utils))`



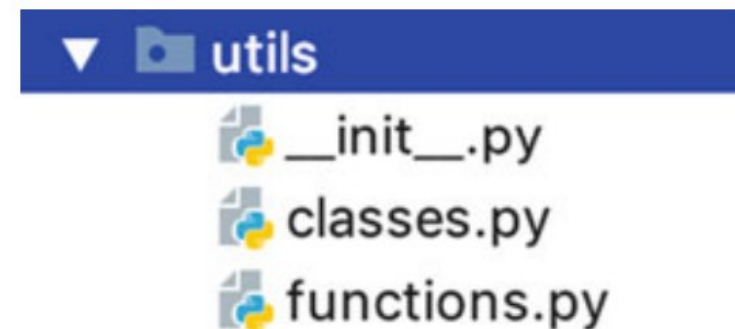
Package

- A package is a collection of modules organized in a directory hierarchy
- It contains an **`__init__.py`** file that signifies the directory as a Python package
- It provides a way to organize related modules and sub-packages
- Examples: *numpy*, *pandas*



Package Organisation

- Python allows developers to organize modules together into packages
 - In a hierarchical structure based on directories
- A package is defined as
 - A directory containing one or more Python source files and
 - An optional source file named **`__init__.py`**
 - This file may also contain code that is executed when a module is imported from the package
- For example, the `utils` package contains two modules: `classes` and `functions`
 - The **`__init__.py`** file contains package level initialisation code
 - The `functions` module then contains several function definitions
 - The `classes` module contains several class definitions



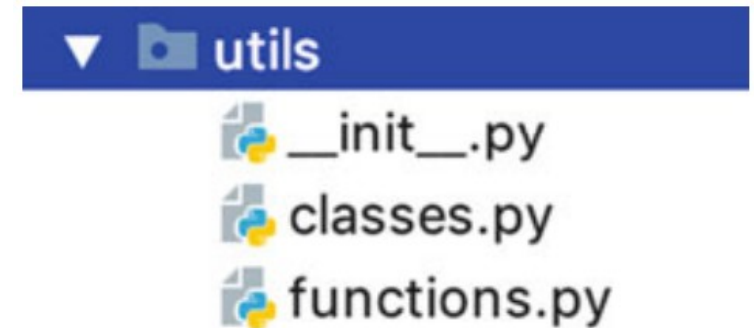


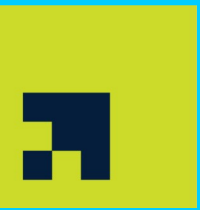
Importing (from) Packages

[Hunt 2019]

12

- We can import individual modules from packages
 - `import utils.functions`
 - `from utils.functions import my_function`
- You can provide some control over what is imported from a package by “from utils import *”
 - You can define a variable `__all__` in the `__init__.py` file
 - `__all__ = ["functions"]`



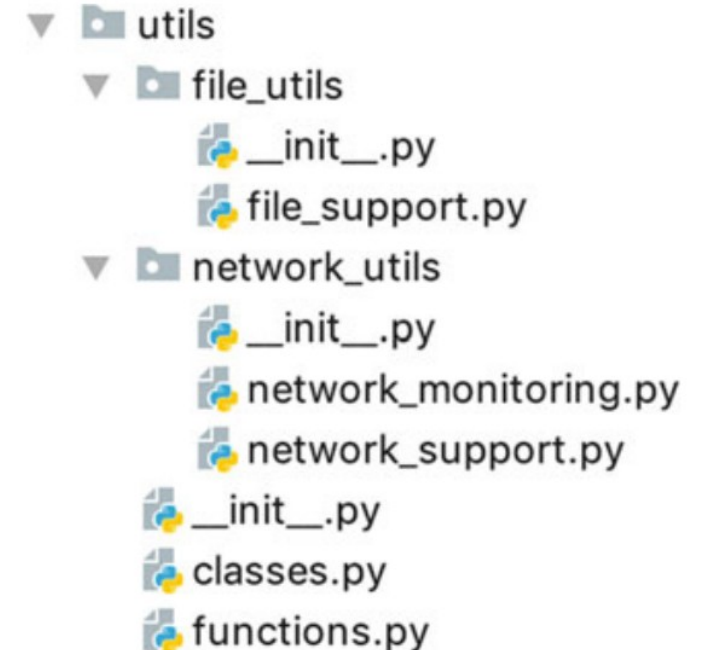


Sub Packages

[Hunt 2019]

13

- Packages can contain sub packages to any depth you require
 - To import the sub package, we do the same as before
 - But each package in the path to the module is separate by a dot
 - `import utils.file_utils.file_support`
 - `from utils.file_utils.file_support import file_logger`





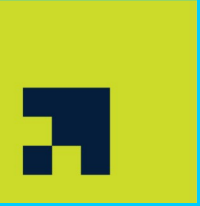
Uploading Your Package to PyPi

<https://medium.com/@joel.barmettler/how-to-upload-your-python-package-to-pypi-65edc5fe9c56>

14

- The PyPi package index is one of the properties that makes python so powerful
 - With just a simple command, you get access to thousands of cool libraries, ready for you to use
- We can upload our packages to PyPi
 - Make your code publish-ready
 - Create a python package
 - Create the files PyPi needs
 - Create a PyPi account
 - Upload your package to github.com
 - Upload your package to PyPi
 - Install your own package using pip
 - Change your package





Some Python Built-in Modules

- **collections**

- Offers specialized container data types

- **csv**

- Delivers functionalities for exporting and importing tabular data in CSV format

- **json**

- JSON encoder and decoder

- **time**

- Time access and conversions

- **datetime**

- Delivers tools that make it easier to work with dates and times



Some Python Built-in Modules

- **math**

- Provides mathematical functions defined by the C standard

- **os**

- Provides functions necessary to interact with the operating system

- **sys**

- Offers access to variables used or maintained by the interpreter

- **random**

- Implements pseudo-random number generators for various distributions

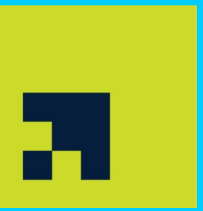
- **re**

- Provides regular expression matching operations



Some Python Third-Party Packages

- NumPy
- Pandas
- SciPy
- Scikit-Learn
- TensorFlow
- Keras



Some Python Third-Party Packages

- Matplotlib
- Seaborn
- Plotly